~/majibill/docs/06-audit-trail.md

$ cat 06-audit-trail.md

MAJIBILL PLATFORM · AUDIT

Audit Trail Design

A tamper-evident, append-only record of who did what, to which record, when, from where, and why — for finance, regulators, tenants and support.

```
hash = sha256(canonical_json(event) || prev_hash)
-- INSERT only; UPDATE/DELETE revoked and trigger-blocked
-- daily Merkle root anchored to object-lock storage
```

DOC	VERSION	DATE
MB-06	1.0 draft	08 Oct 2026

STATUS	AUDIENCE	CLASS
For review	Engineering, security, finance, auditors	Confidential

// Contents

1. Why this matters here . . . 3
2. What is audited . . . 3
3. Event model . . . 4
4. Tamper evidence . . . 4
5. Write path: atomic with the business change . . . 5
6. Action taxonomy (excerpt) . . . 5
7. Controls built on the audit trail . . . 6
8. Audit viewer and exports . . . 6

// 1. Why this matters here

A billing system moves other people's money and decides who gets disconnected. Disputes are certain. The audit trail is the system's memory and its defence: it must show what happened, prove it was not altered, and be readable by a non-engineer.

Audience	Needs
Utility finance / auditors	Trace any balance to bills, payments, adjustments and the people who approved them
Tenant managers	See staff activity, spot misuse, answer customer disputes
Regulators / data protection	Evidence of access control, breach investigation, data-subject request handling
404HubSpot support and security	Investigate incidents; prove support access was authorised and limited
Customers (indirectly)	Statement history and receipts that match the ledger

// 2. What is audited

Class	Examples	Stored in
Business events	Reading accepted/corrected, bill issued/voided, adjustment requested/approved, payment received/reversed, tariff published, write-off, disconnection	`audit_events`
Security events	Login success/failure, MFA changes, role grants, API key created, password reset, session revoked	`audit_events` (category security)
Privileged access	Platform staff support sessions, impersonation, break-glass, data exports	`audit_events` + support-session record
Data access (sensitive)	Viewing national ID, bulk customer export, report downloads with PII	`audit_events` (category access), sampled for routine reads
Integration events	Raw M-Pesa callbacks, SMS delivery receipts, webhook calls	Source tables (`payment_inbox`, `notifications`) linked by id
Configuration	Settings, templates, tariff structure, feature entitlements	`audit_events` with before/after
Infrastructure	Admin console actions, DB role use, deploys	Cloud audit logs + pgaudit, shipped to immutable storage

> NOTE **Two layers, two purposes**
> **Domain audit** (this document) explains business meaning in application terms. **Infrastructure audit** (cloud trail, pgaudit) proves nobody bypassed the application. Both are retained; neither replaces the other.

// 3. Event model

```sql
CREATE TABLE audit_events (
  id            uuid NOT NULL DEFAULT uuid_generate_v7(),
  tenant_id     uuid NOT NULL,
  seq           bigint NOT NULL,                 -- per-tenant, gap-free, assigned in tx
  occurred_at   timestamptz NOT NULL DEFAULT now(),
  category      text NOT NULL,                   -- business|security|access|config|support
  action        text NOT NULL,                   -- 'bill.issued', 'adjustment.approved'
  actor_type    text NOT NULL,                   -- user|system|platform_staff|api_key
  actor_id      uuid,
  on_behalf_of  uuid,                            -- impersonation / support session
  entity_type   text NOT NULL,   entity_id uuid,
  before        jsonb,           after jsonb,    -- field-level, PII-minimised
  reason        text,                            -- mandatory for adjustments, write-offs
  request_id    text, ip inet, user_agent text, device_id text,
  prev_hash     bytea NOT NULL,  hash bytea NOT NULL,
  PRIMARY KEY (tenant_id, occurred_at, id),
  UNIQUE (tenant_id, seq)
) PARTITION BY RANGE (occurred_at);
REVOKE UPDATE, DELETE, TRUNCATE ON audit_events FROM app_role;
```

Field group	Purpose
who: `actor_*`, `on_behalf_of`	Real person or system; support sessions record the platform user and the ticket
what: `action`, `entity_*`, `before/after`	Stable verb-noun taxonomy; changed fields only; sensitive values masked or hashed
when: `occurred_at`, `seq`	Server time (never device time); `seq` makes deletions detectable
where: `ip`, `device_id`, `request_id`	Correlate with logs and traces; mobile device identity for field staff
why: `reason`	Mandatory free text for money-affecting changes; validated non-empty
integrity: `prev_hash`, `hash`	Per-tenant hash chain, see section 4

// 4. Tamper evidence

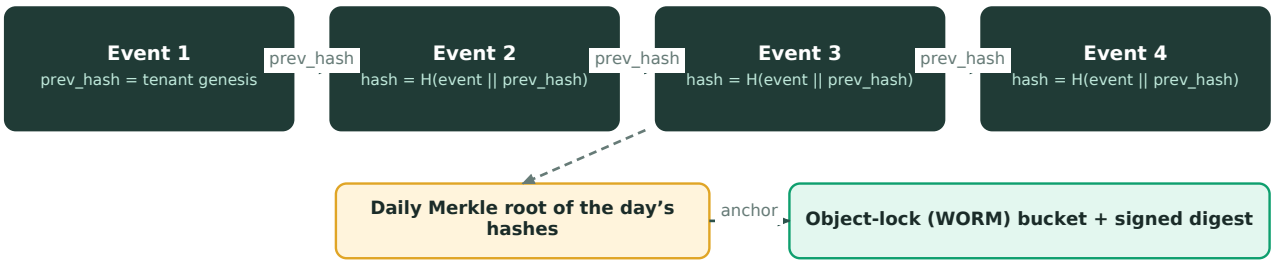


Fig 4.1 Hash chain per tenant, anchored daily to write-once storage outside the database.

- **Detect:** a nightly verifier recomputes the chain and compares the anchors. A changed, removed or reordered row breaks the chain or the `seq` sequence and raises a security alert.
- **Prevent:** the application role has INSERT only; a trigger rejects UPDATE/DELETE even from other roles except a separate, break-glass retention role used only by the partition-detach job.
- **Anchor:** the daily root is written to an object-lock bucket (and optionally emailed to the tenant's auditor). An attacker with database access still cannot rewrite history without breaking the anchor.
- **Verify:** tenants and auditors can run "Verify integrity" in the console for any date range and download a signed verification report.

// 5. Write path: atomic with the business change

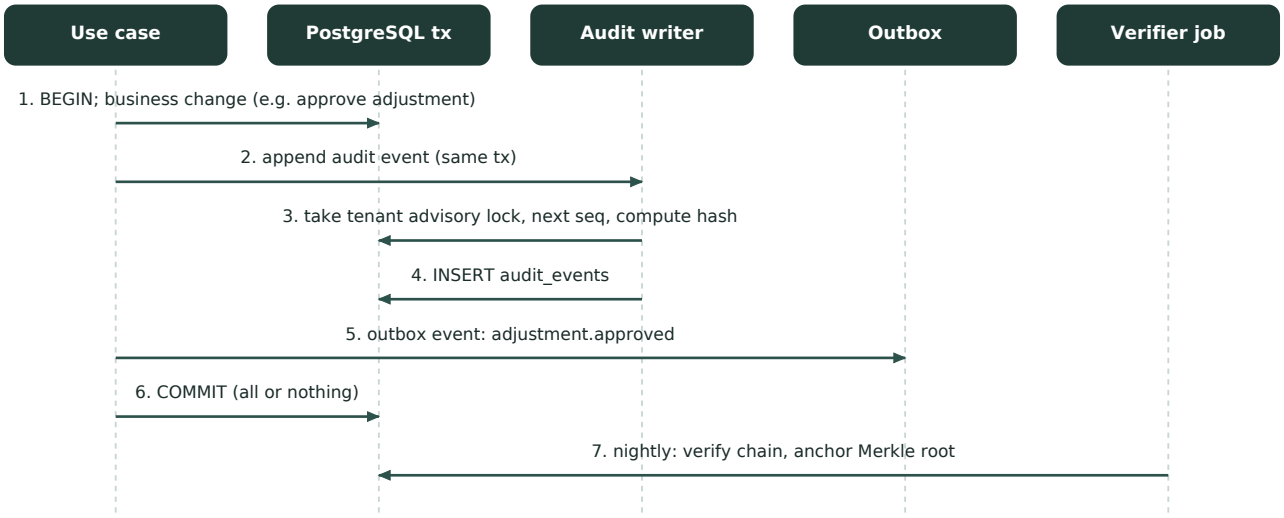


Fig 5.1 If the business change commits, the audit event commits. If either fails, neither exists.

- The audit write shares the transaction, so there is no state where money moved without a record.
- A per-tenant advisory lock serialises chain extension; throughput is ample for the expected rate and can be sharded by day-chain if ever needed.
- Failure policy: if the audit insert fails, the business action fails. Audit is not best-effort.

// 6. Action taxonomy (excerpt)

Domain	Actions
customers	`customer.created`, `customer.updated`, `customer.pii_viewed`, `account.opened`, `account.status_changed`, `account.closed`
meters / readings	`meter.installed`, `meter.replaced`, `reading.accepted`, `reading.flagged`, `reading.corrected`, `reading.estimated`
billing	`cycle.opened`, `cycle.frozen`, `bill.calculated`, `bill.approved`, `bill.issued`, `bill.voided`, `tariff.published`
adjustments	`adjustment.requested`, `adjustment.approved`, `adjustment.rejected`, `writeoff.approved`
payments	`payment.received`, `payment.matched`, `payment.suspended`, `payment.resolved`, `payment.reversed`, `refund.issued`
security	`auth.login`, `auth.failed`, `auth.mfa_enabled`, `role.granted`, `apikey.created`, `session.revoked`
support / platform	`support.session_started`, `support.impersonation_started`, `export.created`, `tenant.suspended`, `tenant.migrated`

// 7. Controls built on the audit trail

7.1 Maker–checker

Action	Maker	Checker	Rule
Bill adjustment above threshold	Billing clerk	Billing supervisor	Different person; reason mandatory
Write-off	Accountant	Finance manager	Policy threshold; documented reason
Tariff publish	Billing manager	Utility manager	Effective date in the future only
Reading correction after cycle freeze	Supervisor	Billing manager	Evidence attached
Manual payment reversal	Cashier	Finance manager	Linked to original payment
Role grant: admin	Tenant admin	Second tenant admin	Notified to all admins

7.2 Support access (break-glass)

1. Platform staff request access with a ticket id and reason; a second approver grants a **time-boxed** session (default 60 min).
2. Session scope is limited (read-only unless escalated); every query path writes `access` events with `on_behalf_of`.
3. The tenant admin sees the session in their own audit viewer in real time and by email.
4. No standing access to tenant data for platform staff; production database access only via the same mechanism.

7.3 Anomaly alerts from audit data

- Many adjustments by one user, or adjustments near cycle close.
- Logins from new countries, impossible travel, repeated failures.
- Bulk exports, large PII views, role grants outside business hours.
- Same approver and requester pattern (collusion hint).
- Chain verification failure, sequence gap, anchor mismatch (critical).

// 8. Audit viewer and exports

Feature	Detail
Timeline	Per customer, account, bill, meter or user; filter by action, actor, date, category
Diff view	Before/after rendered as human-readable changes (“Tariff plan Domestic → Commercial”)
Explain a balance	One click from a balance to the ledger entries and the audit events behind each
Export	CSV/PDF/JSON, itself audited; signed manifest with hash range
Verify integrity	Run chain + anchor verification, download signed report
Access	Roles: tenant auditor (read-only), tenant admin, platform security; PII masked unless permission granted

8.1 Privacy and retention

- Store identifiers and changed field names; mask or hash sensitive values (national ID, full phone) in `before/after`.
- Retain 7 years by default (configurable per tenant, subject to law); anchors kept indefinitely.
- Erasure requests anonymise customer PII in source tables; audit events keep entity ids and actions, with PII fields already minimised, so the chain stays intact. Confirm treatment with counsel (MB-07).

- Audit data itself is tenant data: exportable at tenant exit and deleted with the tenant per contract after the legal retention period.